Modul Hari 6 - Struktur Data I: Array, Objek, Stack, dan Queue

Modul ini memuat teori dan praktik Hari 6 diklat Junior Web Programmer (unit J.620100.004.02 Menggunakan Struktur Data), dengan jadwal 08.00-17.00 = 10 JP (1 JP = 45 menit). Hasil akhirnya adalah implementasi Stack dan Queue serta mini-aplikasi Undo/Redo dan Antrian Loket (bagian 1 dari Proyek 3).

Tujuan Pembelajaran

1. Mengidentifikasi konsep data dan struktur data sesuai konteks permasalahan (KUK 1.1).
2. Membandingkan kelebihan dan kekurangan alternatif struktur data untuk satu masalah (KUK 1.2).
3. Mengimplementasikan array, objek, Map, Set, Stack, dan Queue dalam JavaScript (KUK 2.1).
4. Menyatakan akses data dalam algoritma yang efisien dan menilai kompleksitasnya dengan Big-O (KUK 2.2).
5. Menangani kasus tepi (struktur kosong, satu elemen) pada implementasi struktur data.

Jadwal

Waktu	Kegiatan
08.00-08.45	Review Proyek 2 dan kuis, Teori T1-T2: konsep struktur data dan Big-O
08.45-10.15	Teori T3-T8: array, objek/Map/Set, Stack, Queue, pemilihan struktur data
10.30-12.00	Lab 1: array dan objek untuk data peserta; Lab 2: Map dan Set
13.00-14.30	Lab 3: Stack; Lab 4: Queue; Lab 5: ukur efisiensi + Bug Hunt
14.45-16.15	PBL: mini-aplikasi Undo/Redo dan Antrian Loket (bagian 1 Proyek 3)
16.15-17.00	Code review, dokumentasi, commit, refleksi

Materi Teori

T1. Konsep Data dan Struktur Data

Data adalah fakta mentah (angka, teks, tanggal); **informasi** adalah data yang telah diolah sehingga bermakna. **Struktur data** adalah cara menyimpan dan mengatur data di memori agar operasi tertentu (cari, tambah, hapus, urut) dapat dilakukan secara efisien.

Pembeda	Tipe data	Struktur data
Isi	Jenis satu nilai (number, string, boolean)	Kumpulan banyak nilai dengan aturan akses
Contoh	`42`, `'Sari'`	Array, Stack, Queue, Tree

Klasifikasi:

```
Struktur data
  |-- Linear (elemen berurutan)
  |     |-- Array, Linked List
  |     |-- Stack (LIFO), Queue (FIFO)
  |-- Non-linear (hierarkis/jaringan)
  |     |-- Tree, Graph
  |-- Berbasis kunci
        |-- Objek/Map (kunci -> nilai), Set (nilai unik)
```

Abstract Data Type (ADT) mendefinisikan *apa yang dapat dilakukan* (operasi) tanpa menentukan *bagaimana* dibuat. Contoh: Stack adalah ADT dengan operasi `push`, `pop`, `peek`; ia dapat diimplementasikan dengan array atau linked list.

Langkah memilih struktur data (KUK 1.1):

1. Tuliskan masalah dan data yang terlibat.
2. Daftarkan operasi yang dibutuhkan dan **mana yang paling sering** (cari, tambah, hapus, urut, ambil terakhir/pertama).
3. Tentukan batasan (jumlah data, perlu urutan, perlu unik, perlu akses berdasar kunci).
4. Bandingkan 2-3 kandidat (KUK 1.2), lalu pilih dan jelaskan alasannya.

T2. Efisiensi dan Notasi Big-O

Program yang benar belum tentu efisien. Efisiensi diukur dari **waktu** (jumlah langkah) dan **ruang** (memori) terhadap ukuran data **n**. **Big-O** menggambarkan laju pertumbuhan langkah pada kasus terburuk.

Notasi	Nama	Contoh	Langkah untuk n = 1.000.000
O(1)	Konstan	`arr[5]`, `push`, `pop`	1

Notasi	Nama	Contoh	Langkah untuk n = 1.000.000
O(log n)	Logaritmik	Binary search	sekitar 20
O(n)	Linear	Satu loop, linear search	1.000.000
O(n log n)	Linearitmik	Merge sort	sekitar 20.000.000
O(n^2)	Kuadratik	Dua loop bersarang	1.000.000.000.000

```
// O(1): jumlah langkah tidak bergantung pada n
const pertama = daftar[0];

// O(n): satu loop
for (const x of daftar) { jumlah += x; }

// O(n^2): loop bersarang
for (const a of daftar) {
  for (const b of daftar) { /* bandingkan a dan b */ }
}
```

Aturan menghitung: abaikan konstanta (2n menjadi n), ambil suku terbesar (n^2 + n menjadi n^2), loop bersarang dikalikan. Mengukur nyata: `console.time('label')` dan `console.timeEnd('label')`.

T3. Array

Array menyimpan elemen berurutan dengan indeks mulai 0. Di JavaScript array dinamis (ukuran dapat berubah).

Operasi	Kompleksitas	Catatan
Akses `arr[i]`	O(1)	Cepat berdasarkan indeks
Cari nilai (`indexOf`, `includes`)	O(n)	Periksa elemen satu per satu
`push` / `pop` (ujung akhir)	O(1)	Murah
`shift` / `unshift` (ujung awal)	O(n)	Semua elemen bergeser
Sisip/hapus di tengah (`splice`)	O(n)	Elemen sesudahnya bergeser

Kelebihan: akses indeks cepat, sederhana, hemat memori. Kekurangan: sisip/hapus di awal atau tengah mahal, pencarian nilai lambat bila data besar dan tidak berurutan.

T4. Objek, Map, dan Set

```
// Objek: pasangan kunci-nilai (kunci berupa string)
const peserta = { id: 1, nama: 'Sari', kelas: 'Web', aktif: true };
peserta.nama;           // akses titik
peserta['kelas'];       // akses kurung
peserta.email = 'sari@contoh.com';   // tambah
delete peserta.aktif;                // hapus
'nama' in peserta;                   // true
Object.keys(peserta);                // ['id', 'nama', 'kelas', 'email']

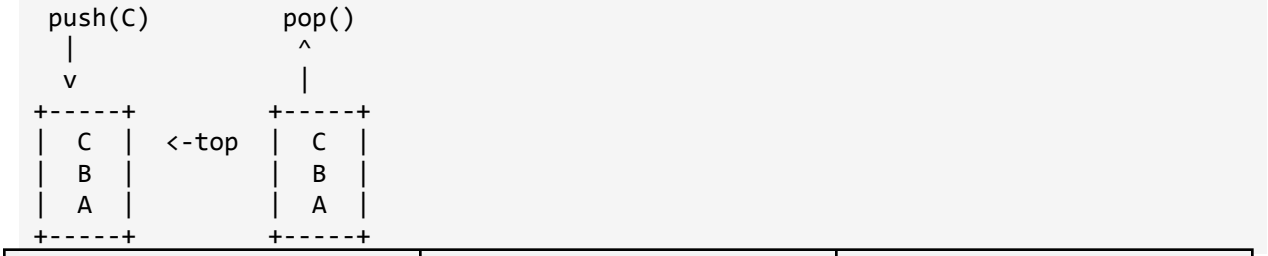
// Map: pasangan kunci-nilai, kunci bertipe apa saja, urutan terjaga
const nilai = new Map();
nilai.set('Sari', 90);
nilai.set('Budi', 68);
nilai.get('Sari');      // 90
nilai.has('Budi');      // true
nilai.delete('Budi');
nilai.size;             // 1

// Set: kumpulan nilai unik
const email = new Set(['a@x.com', 'b@x.com', 'a@x.com']);
email.size;             // 2
email.has('b@x.com');   // true
const unik = [...email];
```

Struktur	Kunci	Pencarian	Dipakai untuk
Objek	String/simbol	O(1) rata-rata	Satu entitas bersifat tetap (peserta, konfigurasi)
Map	Tipe apa saja	O(1) rata-rata	Kamus dinamis, penghitung frekuensi
Set	Nilai itu sendiri	O(1) rata-rata	Menghilangkan duplikat, uji keanggotaan cepat

T5. Stack (LIFO)

Stack (tumpukan) mengikuti prinsip **LIFO (Last In, First Out)**: elemen terakhir yang masuk adalah yang pertama keluar, seperti tumpukan piring.



Operasi	Fungsi	Kompleksitas
`push(x)`	Menaruh di atas	O(1)
`pop()`	Mengambil dan menghapus elemen teratas	O(1)
`peek()`	Melihat elemen teratas tanpa menghapus	O(1)
`isEmpty()`	Apakah kosong	O(1)
`size()`	Jumlah elemen	O(1)

Contoh penggunaan: undo/redo editor, tombol Back browser, call stack pemanggilan fungsi, validasi tanda kurung, evaluasi ekspresi postfix, penelusuran DFS.

```
class Stack {
  constructor() {
    this.items = [];
  }

  push(item) {
    this.items.push(item);
  }

  pop() {
    return this.isEmpty() ? undefined : this.items.pop();
  }

  peek() {
    return this.isEmpty() ? undefined : this.items[this.items.length - 1];
  }

  isEmpty() {
    return this.items.length === 0;
  }
```

```
  size() {
    return this.items.length;
  }

  clear() {
    this.items = [];
  }
}
```

T6. Queue (FIFO)

Queue (antrean) mengikuti prinsip **FIFO (First In, First Out)**: elemen yang pertama masuk adalah yang pertama keluar, seperti antrean loket.

```
  dequeue <-- [ A | B | C | D ] <-- enqueue
   (depan)                          (belakang)
```

Operasi	Fungsi
`enqueue(x)`	Menambah di belakang
`dequeue()`	Mengambil dan menghapus elemen terdepan
`front()`	Melihat elemen terdepan
`isEmpty()`, `size()`	Status antrean

Implementasi sederhana memakai `array.shift()` bernilai **O(n)**. Versi efisien memakai objek dengan penanda `head` dan `tail` sehingga semua operasi O(1):

```
class Queue {
  constructor() {
    this.items = {};
    this.head = 0;
    this.tail = 0;
  }

  enqueue(item) {
    this.items[this.tail] = item;
    this.tail++;
  }

  dequeue() {
    if (this.isEmpty()) return undefined;
    const item = this.items[this.head];
    delete this.items[this.head];
    this.head++;
    return item;
  }

  front() {
    return this.items[this.head];
  }
```

```
  isEmpty() {
    return this.tail === this.head;
  }

  size() {
    return this.tail - this.head;
  }
}
```

Variasi: *priority queue* (yang prioritas tinggi keluar dulu, mis. IGD), *deque* (masuk dan keluar dari dua ujung), *circular queue* (memakai ulang ruang). Contoh penggunaan: antrean loket, antrean cetak, penjadwalan tugas, penelusuran BFS.

T7. Membandingkan dan Memilih Struktur Data (KUK 1.2)

Kebutuhan utama	Pilihan	Alasan	Kekurangan
Akses berdasar urutan/indeks, daftar tetap	Array	Akses O(1), sederhana	Sisip tengah O(n)
Cari berdasar kunci (id, nama)	Objek/Map	Pencarian O(1) rata-rata	Tidak punya urutan numerik
Memastikan nilai unik, uji keanggotaan	Set	`has` O(1), otomatis unik	Tanpa indeks
Operasi terakhir diproses duluan (undo)	Stack	`push`/`pop` O(1)	Akses elemen tengah sulit
Layanan adil berdasar urutan datang	Queue	`enqueue`/`dequeue` O(1)	Akses tengah sulit
Sering sisip/hapus di awal	Linked List (Hari 7)	O(1) di kepala	Akses indeks O(n)

Templat analisis (dikumpulkan di PBL):

Bagian	Isi
Masalah	Deskripsi singkat
Data	Jenis dan perkiraan jumlah
Operasi dominan	Daftar operasi dan frekuensi
Alternatif	Minimal dua struktur data
Perbandingan	Kompleksitas, kelebihan, kekurangan

Bagian	Isi
Keputusan	Struktur yang dipilih dan alasan

T8. Mengakses Data dengan Efisien (KUK 2.2)

Mengganti struktur data sering lebih berpengaruh daripada mengoptimalkan baris kode.

```
// Lambat: O(n^2) - memeriksa duplikat dengan loop bersarang
function adaDuplikatLambat(daftar) {
  for (let i = 0; i < daftar.length; i++) {
    for (let j = i + 1; j < daftar.length; j++) {
      if (daftar[i] === daftar[j]) return true;
    }
  }
  return false;
}

// Cepat: O(n) - memakai Set
function adaDuplikat(daftar) {
  const terlihat = new Set();
  for (const x of daftar) {
    if (terlihat.has(x)) return true;
    terlihat.add(x);
  }
  return false;
}

// Menghitung frekuensi dengan Map: O(n)
function hitungFrekuensi(kata) {
  const frekuensi = new Map();
  for (const k of kata) {
    frekuensi.set(k, (frekuensi.get(k) || 0) + 1);
  }
  return frekuensi;
}
```

Prinsip: gunakan indeks atau kunci untuk akses langsung, hindari pencarian berulang di array tidak terurut, dan manfaatkan `Set`/`Map` untuk uji keanggotaan.

Praktik Hari 6

Lab 1 - Array dan Objek (Data Peserta)

Buat `hari-06-struktur-data/lab1.js` dengan data berikut (tambahkan sampai 10 peserta):

```
const peserta = [
  { id: 1, nama: 'Sari', kelas: 'Web', nilai: 90, aktif: true },
  { id: 2, nama: 'Budi', kelas: 'Web', nilai: 68, aktif: true },
  { id: 3, nama: 'Citra', kelas: 'Data', nilai: 82, aktif: false }
];
```

Tulis fungsi dan uji pada tiap soal:

1. Tampilkan peserta aktif saja (`filter`).
2. Urutkan berdasarkan nama (`sort` dengan `localeCompare`) tanpa mengubah array asli.
3. Cari peserta berdasarkan `id` (`find`) dan tangani bila tidak ditemukan.
4. Hitung jumlah peserta per kelas dengan objek penghitung.
5. Tambah, ubah, dan hapus peserta (`push`, perubahan properti, `filter` atau `splice`) lalu cetak hasilnya.
6. Ubah array menjadi objek berindeks id (`reduce`) sehingga pencarian berdasar id menjadi O(1).

Lab 2 - Map dan Set

1. Hitung frekuensi kata dari kalimat `'saya belajar web saya suka web'` memakai `Map`, lalu tampilkan terurut dari yang paling sering.
2. Hilangkan duplikat dari daftar email memakai `Set`, tampilkan sebelum dan sesudah.
3. Cari irisan dua daftar peserta (kelas A dan kelas B) memakai `Set`.
4. Buat fungsi `adaDuplikat(daftar)` (versi `Set`) dan uji dengan array kosong, satu elemen, dan elemen berulang.

Lab 3 - Stack

1. Buat `structures.js` dan salin kelas `Stack` (T5) dengan `export class Stack`.
2. Uji dengan `push` tiga elemen, `peek`, `pop` sampai kosong, lalu `pop` pada stack kosong (tidak boleh error).
3. Implementasikan validasi tanda kurung seimbang:

```
import { Stack } from './structures.js';

export function kurungSeimbang(teks) {
  const pasangan = { ')': '(', ']': '[', '}': '{' };
  const tumpukan = new Stack();

  for (const karakter of teks) {
    if ('([{'.includes(karakter)) {
      tumpukan.push(karakter);
    } else if (karakter in pasangan) {
      if (tumpukan.isEmpty() || tumpukan.pop() !== pasangan[karakter]) {
        return false;
      }
    }
  }
  return tumpukan.isEmpty();
}

console.log(kurungSeimbang('(a+b)*[c-d]'));  // true
console.log(kurungSeimbang('(a+b]'));        // false
console.log(kurungSeimbang('((a)'));         // false
console.log(kurungSeimbang(''));             // true
```

4. Balik sebuah string memakai Stack, dan periksa palindrom.

5. Jalankan dengan Node: ubah `package.json` menambahkan `"type": "module"` atau gunakan ekstensi `.mjs`, atau uji lewat halaman HTML dengan `<script type='module'>`.

Lab 4 - Queue dan Simulasi Antrean Loket

1. Tambahkan kelas `Queue` (T6, versi `head` dan `tail`) ke `structures.js`.
2. Uji dengan `enqueue` tiga elemen, `dequeue` hingga kosong, dan `dequeue` pada queue kosong.
3. Buat halaman `antrian.html` dengan tombol Ambil Nomor dan Panggil:

```
<h1>Antrean Loket</h1>
<p>Sedang dipanggil: <strong id='dipanggil'>-</strong></p>
<p>Menunggu: <span id='jumlah'>0</span></p>
<button id='ambil'>Ambil Nomor</button>
<button id='panggil'>Panggil Berikutnya</button>
<ol id='daftar'></ol>
<script type='module' src='js/antrian.js'></script>
```

```
import { Queue } from './structures.js';

const antrian = new Queue();
let nomorTerakhir = 0;

const dipanggil = document.getElementById('dipanggil');
const jumlah = document.getElementById('jumlah');
const daftar = document.getElementById('daftar');

function formatNomor(angka) {
  return 'A' + String(angka).padStart(3, '0');
}

function tampilkanDaftar() {
  daftar.textContent = '';
  for (let i = antrian.head; i < antrian.tail; i++) {
    const li = document.createElement('li');
    li.textContent = antrian.items[i];
    daftar.append(li);
  }
  jumlah.textContent = antrian.size();
}

document.getElementById('ambil').addEventListener('click', () => {
  nomorTerakhir++;
  antrian.enqueue(formatNomor(nomorTerakhir));
  tampilkanDaftar();
});

document.getElementById('panggil').addEventListener('click', () => {
  dipanggil.textContent = antrian.isEmpty() ? 'Antrean kosong' : antrian.dequeue();
  tampilkanDaftar();
});
```

Catatan desain: `tampilkanDaftar` membaca properti internal `head`, `tail`, dan `items`. Itu melanggar enkapsulasi; sebagai tantangan, tambahkan method `toArray()` pada `Queue` dan pakai itu.

Lab 5 - Mengukur Efisiensi

```
const N = 100000;
const angka = Array.from({ length: N }, (_, i) => i);
const himpunan = new Set(angka);

console.time('Array.includes x 1000');
for (let i = 0; i < 1000; i++) angka.includes(N - 1);
console.timeEnd('Array.includes x 1000');

console.time('Set.has x 1000');
for (let i = 0; i < 1000; i++) himpunan.has(N - 1);
console.timeEnd('Set.has x 1000');

const arr = Array.from({ length: N }, (_, i) => i);
console.time('Array.shift');
while (arr.length > 0) arr.shift();
console.timeEnd('Array.shift');

const q = new Queue();
for (let i = 0; i < N; i++) q.enqueue(i);
console.time('Queue.dequeue');
while (!q.isEmpty()) q.dequeue();
console.timeEnd('Queue.dequeue');
```

Isi tabel hasil pengukuran (jalankan tiga kali dan ambil rata-rata), lalu jelaskan selisihnya dengan Big-O:

Operasi	Waktu (ms)	Big-O
`Array.includes` x 1000		O(n) per pencarian
`Set.has` x 1000		O(1) per pencarian
`Array.shift` hingga kosong		O(n) per operasi
`Queue.dequeue` hingga kosong		O(1) per operasi

Bug Hunt Hari 6

Instruktur membagikan `bug-struktur.js` berisi 6 kesalahan.

No	Kode bermasalah	Gejala	Perbaikan
1	`dequeue()` pada queue kosong hasilnya dipakai sebagai angka	`undefined` atau `NaN` pada tampilan	Periksa `isEmpty()` sebelum memakai hasil

No	Kode bermasalah	Gejala	Perbaikan
2	`pop()` stack kosong tanpa pemeriksaan	`undefined` merambat ke logika lain	Kembalikan nilai aman dan beri pesan
3	`peek()` memakai `items[items.length]`	Selalu `undefined`	`items[items.length - 1]`
4	`size()` pada Queue menghitung `tail + head`	Ukuran salah	`tail - head`
5	`const salinan = daftar; salinan.sort()`	Array asli ikut berubah	`[...daftar].sort(...)`
6	Mencari id dengan `find` di dalam loop 100.000 kali	Lambat, O(n^2)	Buat Map/objek berindeks id lebih dulu

PBL Hari 6 - Undo/Redo dan Antrean Loket (Bagian 1 Proyek 3)

Studi kasus: sebuah kantor kecil membutuhkan dua alat sederhana: editor catatan yang dapat membatalkan dan mengulang pengetikan, serta layar antrean loket. Hari ini peserta membangun versi dasar; Hari 7 melanjutkan dengan Linked List, penambahan fitur, dan penerapan guideline (Proyek 3).

Fitur A - Editor Undo/Redo (dua Stack)

```
<textarea id='editor' rows='6' cols='50'></textarea>
<p>
  <button id='undo' disabled>Undo</button>
  <button id='redo' disabled>Redo</button>
  <span id='info'></span>
</p>
<script type='module' src='js/editor.js'></script>
```

```
import { Stack } from './structures.js';

const editor = document.getElementById('editor');
const tombolUndo = document.getElementById('undo');
const tombolRedo = document.getElementById('redo');
const info = document.getElementById('info');

const undoStack = new Stack();
const redoStack = new Stack();
let teksSekarang = '';

function perbaruiTombol() {
```

```
  tombolUndo.disabled = undoStack.isEmpty();
  tombolRedo.disabled = redoStack.isEmpty();
  info.textContent = `Riwayat: ${undoStack.size()} undo, ${redoStack.size()} redo`;
}

editor.addEventListener('input', () => {
  undoStack.push(teksSekarang);
  teksSekarang = editor.value;
  redoStack.clear();
  perbaruiTombol();
});

tombolUndo.addEventListener('click', () => {
  if (undoStack.isEmpty()) return;
  redoStack.push(teksSekarang);
  teksSekarang = undoStack.pop();
  editor.value = teksSekarang;
  perbaruiTombol();
});

tombolRedo.addEventListener('click', () => {
  if (redoStack.isEmpty()) return;
  undoStack.push(teksSekarang);
  teksSekarang = redoStack.pop();
  editor.value = teksSekarang;
  perbaruiTombol();
});

perbaruiTombol();
```

Catatan: menyimpan salinan teks setiap ketukan boros memori. Tantangan: batasi riwayat maksimal 50 langkah dan simpan snapshot tiap jeda mengetik (*debounce*).

Fitur B - Antrean Loket (Queue) memakai Lab 4, dengan tambahan: nomor di-reset tiap hari, riwayat 5 nomor terakhir yang sudah dipanggil, dan tombol Reset.

Deliverable Hari 6:

1. Berkas `structures.js` (Stack dan Queue) dengan komentar fungsi.
2. Editor Undo/Redo dan Antrean Loket berjalan.
3. **Dokumen Analisis Pemilihan Struktur Data** (templat T7) untuk tiga studi kasus berikut:

Studi kasus	Pertanyaan analisis
Daftar kontak ponsel dengan pencarian nama	Struktur apa untuk pencarian cepat berdasarkan nama?
Riwayat halaman yang dikunjungi (tombol Back)	Mengapa Stack dan bagaimana tombol Forward bekerja?

Studi kasus	Pertanyaan analisis
Antrean pasien klinik dengan kasus darurat	Mengapa Queue biasa tidak cukup dan alternatif apa yang dibutuhkan?

Rubrik (100 poin):

Aspek	Bobot	Indikator
Implementasi Stack dan Queue	30	Operasi benar, kasus tepi tertangani
Aplikasi berjalan	25	Undo/Redo dan Antrean Loket sesuai spesifikasi
Analisis pemilihan struktur data	25	Operasi dominan, perbandingan, keputusan beralasan
Efisiensi	10	Memakai versi O(1) dan memahami Big-O
Kebersihan kode dan Git	10	Penamaan, komentar, commit bermakna

Code Review, Dokumentasi, dan Commit

Butir	Ya/Tidak	Catatan
Struktur data dan UI dipisah berkas (modul)		
Struktur kosong dan satu elemen sudah diuji		
Tidak ada akses langsung ke properti internal dari luar kelas		
Nama method konsisten (`push`, `pop`, `enqueue`, `dequeue`)		
Alasan pemilihan struktur data tertulis		

```
git add .
git commit -m 'Hari 6: Stack, Queue, undo/redo, antrean loket'
git push
```

Bukti Portofolio Hari 6

1. `structures.js`, `kurungSeimbang`, dan hasil uji.
2. Editor Undo/Redo serta Antrean Loket (tangkapan layar).
3. Tabel pengukuran efisiensi (Lab 5) dan dokumen Analisis Pemilihan Struktur Data.
4. Tabel Bug Hunt.

Evaluasi Hari 6

Kuis tertulis (10 soal):

1. Prinsip yang dipakai Stack adalah ... (A. FIFO; B. LIFO; C. Random; D. Priority).
2. Operasi yang melihat elemen teratas tanpa menghapus adalah ... (A. `pop`; B. `push`; C. `peek`; D. `clear`).
3. Kompleksitas akses `arr[i]` adalah ... (A. O(1); B. O(n); C. O(log n); D. O(n^2)).
4. Mengapa `Array.shift()` lambat untuk antrean besar? (A. Menghapus memori; B. Semua elemen digeser sehingga O(n); C. Mengurutkan array; D. Menyalin objek).
5. Struktur data yang otomatis menolak nilai ganda adalah ... (A. Array; B. Stack; C. Set; D. Queue).
6. Struktur terbaik untuk fitur Undo adalah ... (A. Queue; B. Stack; C. Set; D. Tree).
7. Struktur terbaik untuk antrean loket adalah ... (A. Stack; B. Set; C. Queue; D. Objek).
8. Dua loop bersarang pada n data umumnya berkompleksitas ... (A. O(1); B. O(n); C. O(n^2); D. O(log n)).
9. Untuk mencari data berdasar id dengan cepat sebaiknya ... (A. loop `for` tiap kali; B. Map atau objek berindeks id; C. `sort` dulu setiap kali; D. Stack).
10. Hasil `kurungSeimbang('(a+b]')` adalah ... (A. true; B. false; C. error; D. undefined).

Kunci: 1-B, 2-C, 3-A, 4-B, 5-C, 6-B, 7-C, 8-C, 9-B, 10-B.

Pertanyaan lisan: jelaskan mengapa memilih struktur data memengaruhi kecepatan program dan berikan satu contoh dari proyek hari ini.

Ceklis keluaran:

- ☐ Stack dan Queue lolos uji termasuk kasus kosong.
- ☐ Undo/Redo dan Antrean Loket berfungsi.
- ☐ Tabel efisiensi dan Analisis Pemilihan Struktur Data terisi.
- ☐ Kode modular dan lolos code review.
- ☐ Commit dan push selesai.